

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 1 | SLO 1: Elements of Programming Languages (Code: 1011)

Q: 1. What is the syntax and semantics of the following Java statement: `int x = 5 + 3;`?

Ans: Syntax: The statement consists of a type specifier (`int`), an identifier variable name (`x`), the assignment operator (`=`), an arithmetic expression (`5 + 3`), and a statement terminator semicolon (`;`). It complies strictly with Java's Context-Free Grammar (CFG) rules for local variable declaration and initialization.

Semantics: At runtime, memory is allocated on the stack for a 32-bit signed two's complement integer. The binary addition operator computes the arithmetic sum of operands 5 and 3, yielding 8. The value 8 is then stored in the memory location referenced by variable `x`.

Q: 2. Identify lexical tokens in a simple Java program.

Ans: Consider the statement: `int count = 10;`

- **Keyword:** `int` (reserved word defining data type)
- **Identifier:** `count` (user-defined variable name)
- **Operator:** `=` (assignment operator)
- **Literal:** `10` (integer constant literal)
- **Separator/Punctuator:** `;` (statement terminator delimiter)

Q: 3. Modify a sample program to demonstrate the use of grammar rules in Java.

Ans: Java Demonstration:

Grammar rules dictate that statements must be inside methods, variables must be typed, and block scopes must be balanced.

```
public class GrammarDemo {
    public static void main(String[] args) {
        // Rule 1: Type declaration preceding identifier
        int base = 10;
        int height = 5;
        // Rule 2: Valid arithmetic expression and semicolon termination
        double area = 0.5 * base * height;
        // Rule 3: Balanced curly braces for block scope
        if (area > 0) {
            System.out.println("Calculated Area: " + area);
        }
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 1 | SLO 2: Syntax and Semantics (Code: 1012)

Q: 1. Which of the following is not an element of a programming language?

a) Syntax b) Algorithm c) Semantics d) Data types

Ans: Answer: b) Algorithm

Explanation: An algorithm is a language-independent step-by-step logic or procedure to solve a problem. In contrast, Syntax, Semantics, and Data Types are core formal elements inherent to programming language specifications.

Q: 2. Identify the correct pair of syntax and semantics:

a) Brackets → Loop b) Colon → Meaning c) if-else → Logic rule d) Semicolon → Statement terminator

Ans: Answer: d) Semicolon → Statement terminator

Explanation: The semicolon (;) is the concrete syntax in C/C++/Java, and its formal semantic meaning is marking the termination of an execution statement.

Q: 3. True or False: The semantics of a language define its grammatical rules.

Ans: Answer: False

Explanation: Grammatical rules define how valid expressions and statements are composed, which is the domain of **Syntax**. **Semantics** defines the meaning, runtime behavior, and evaluation of syntactically valid constructs.

Q: 4. What is the element responsible for variable declaration and type enforcement?

Ans: Answer: Data Type System / Type System

Explanation: The type system governs variable declarations, sets memory allocation requirements, defines permissible operations, and enforces compile-time and runtime type safety constraints.

Q: 5. Which element determines how functions and control flows are structured?

Ans: Answer: Control Structures and Formal Grammar (Control Flow Constructs)

Explanation: Control structures (such as conditional branching, loops, and subroutine call/return mechanisms) dictate the execution path and instruction sequencing of a program.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 2 | SLO 1: Programming Language Theory (Code: 1021)

Q: 1. Syntax Check: Find and fix the syntax error in this Java code:

```
public class Test {  
    public static void main(String[] args) {  
        int num = 10  
        System.out.println(num);  
    }  
}
```

Ans: Error: Missing statement terminator semicolon (;) after int num = 10 on line 3.

Corrected Code:

```
public class Test {  
    public static void main(String[] args) {  
        int num = 10; // Fixed: added semicolon  
        System.out.println(num);  
    }  
}
```

Q: 2. Semantics Understanding: What will be the output of this code, and why?

```
int x = 5;  
int y = x;  
x = 10;  
System.out.println(y);
```

Ans: Output: 5

Explanation: In Java, primitive data types (like int) are passed and copied entirely by value. When int y = x; executes, the bitwise value of x (which is 5) is copied directly into y's distinct memory location. Modifying x = 10; afterwards has zero effect on y.

Q: 3. Pragmatics Insight: In Java, why is using ArrayList generally preferred over arrays for dynamic collections in modern programming environments?

Ans: Explanation:

- **Dynamic Resizing:** Standard Java arrays have fixed capacity defined at instantiation. ArrayList dynamically grows its internal storage buffer automatically as elements are appended.
- **Rich Utility Methods:** Provides built-in methods like add(), remove(), contains(), indexOf(), and clear().
- **Generics & Type Safety:** Enforces compile-time type checking with generic parameters (e.g., ArrayList<String>).
- **Interoperability:** Integrates seamlessly with the Java Collections Framework, iterators, and Java Stream API for functional filtering and mapping.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 2 | SLO 2: Abstraction & Specification (Code: 1022)

Q: 1. Conceptual: Why is strong typing considered a significant design choice in modern programming languages?

Ans: Explanation: Strong typing ensures that the language strictly forbids implicit, unsafe type conversions (coercions) that could corrupt data or lead to unexpected behavior. It enhances code correctness, memory safety, predictability, and prevents subtle runtime bugs by forcing explicit conversions.

Q: 2. Analytical: Compare static typing and dynamic typing. How do these affect error detection and program reliability?

Ans: Comparison:

- **Static Typing (e.g., Java, C++):** Types are bound and verified at compile time. Syntax and type mismatch bugs are caught early before execution. IDEs provide accurate autocompletion and refactoring, yielding higher software reliability for enterprise systems.
- **Dynamic Typing (e.g., Python, JavaScript):** Types are associated with runtime values rather than variables. Offers rapid prototyping and concise syntax, but type errors surface only during runtime execution, requiring extensive test suite coverage.

Q: 3. Critical Thinking: Java and Python use different syntax styles. How does syntax design impact programmer productivity and code maintainability?

Ans: Analysis:

- **Python's Syntax:** Relies on meaningful indentation and omission of semicolons/parentheses. This minimizes visual boilerplate, speeding up initial development and prototyping.
- **Java's Syntax:** Utilizes explicit C-family braces ({}) and mandatory type declarations. While more verbose, it provides clear, unambiguous structural boundaries that facilitate maintainability and large-scale multi-developer collaboration.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 3 | SLO 1: Structured Programming & Bohm-Jacopini (Code: 1031)

Q: 1. Conceptual: Explain the three control structures identified in the Böhm-Jacopini theorem with simple Java examples.

Ans: The Böhm-Jacopini theorem proved that any computable function can be implemented using only three primary control structures:

1. **Sequence:** Ordered, step-by-step linear execution.
2. **Selection:** Conditional branching between paths (e.g., if-else or switch).
3. **Iteration:** Repetitive loop execution while a boolean predicate holds true (e.g., while, for).

```
// 1. Sequence
int a = 10;
int b = 20;
int sum = a + b;

// 2. Selection
if (sum > 25) {
    System.out.println("Sum is large");
} else {
    System.out.println("Sum is normal");
}

// 3. Iteration
for (int i = 0; i < 3; i++) {
    System.out.println("Iteration: " + i);
}
```

Q: 2. Analytical: Why did the Böhm-Jacopini theorem lead to the decline of the goto statement in modern programming languages?

Ans: Explanation: Prior to structured programming, code heavily relied on arbitrary jumps (goto), resulting in unstructured 'spaghetti code' that was impossible to trace, debug, or mathematically verify. Böhm-Jacopini provided the mathematical proof that goto is completely unnecessary, directly inspiring structured control statements with single-entry and single-exit points.

Q: 3. Application: Rewrite the following pseudocode (which uses goto) using structured constructs:

start:

x = x + 1

if (x < 10) goto start

Ans: Structured Equivalent in Java:

This construct executes the body at least once, making a do-while loop the most idiomatic equivalent:

```
int x = 0;
do {
    x = x + 1;
} while (x < 10);

// Alternatively using a standard while loop:
while (x < 10) {
    x = x + 1;
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 3 | SLO 2: Apply Control Structures (Code: 1032)

Q: 1. Sequence: Write a program to print your name and age.

Ans: Java Implementation:

```
public class SequenceDemo {  
    public static void main(String[] args) {  
        String name = "Sautrik Roy";  
        int age = 20;  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
    }  
}
```

Q: 2. Selection (if-else): Write code to check if a number is even or odd.

Ans: Java Implementation:

```
public class EvenOddDemo {  
    public static void main(String[] args) {  
        int number = 17;  
        if (number % 2 == 0) {  
            System.out.println(number + " is Even");  
        } else {  
            System.out.println(number + " is Odd");  
        }  
    }  
}
```

Q: 3. Iteration (loop): Print numbers from 1 to 5 using a loop.

Ans: Java Implementation:

```
public class LoopDemo {  
    public static void main(String[] args) {  
        System.out.println("Numbers from 1 to 5:");  
        for (int i = 1; i <= 5; i++) {  
            System.out.print(i + " ");  
        }  
        System.out.println();  
    }  
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 4 | SLO 1: Multiple Programming Paradigms (Code: 1041)

Q: 1. What is a programming paradigm?

Ans: Definition: A programming paradigm is a fundamental classification, overarching style, or architectural methodology of computer programming that defines how computation is structured and organized.

Q: 2. Name any three common programming paradigms.

Ans: 1. **Imperative / Procedural Paradigm** (e.g., C)
2. **Object-Oriented Paradigm (OOP)** (e.g., Java, C++)
3. **Functional Paradigm** (e.g., Haskell, Scala, Lisp)

Q: 3. Why is it useful for a language to support multiple paradigms?

Ans: Utility: Multi-paradigm languages (like modern Java, Python, and C++) allow developers to apply the optimal approach for each distinct task. For example, using declarative functional streams for high-speed data transformations while using object-oriented classes for complex business domain modeling.

Q: 4. Which paradigm focuses on real-world entities like objects and classes?

Ans: Answer: Object-Oriented Programming (OOP) Paradigm.

Q: 5. Which paradigm emphasizes step-by-step instructions to change program state?

Ans: Answer: Imperative Programming Paradigm.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 4 | SLO 2: Paradigm Hierarchy & Features (Code: 1042)

Q: 1. Conceptual: Arrange the following in a correct hierarchy:

• Imperative • Structured • Procedural • Object-Oriented

Ans: Correct Architectural Hierarchy:

1. **Imperative Programming** (Broadest category: state-manipulation paradigm)
2. ■■■ **Structured Programming** (Sub-discipline: eliminates goto using sequence, selection, iteration)
3. ■■■ **Procedural Programming** (Modular extension: groups instructions into routines/procedures)
4. ■■■ **Object-Oriented Programming** (Modern evolution: bundles state and behavior into encapsulated objects)

Q: 2. Definition Match: Match each paradigm to its defining feature:

• Functional • Object-Oriented • Procedural • Logic-Based

- a) Uses facts and rules for computation
- b) Focuses on objects and classes
- c) Uses functions to avoid mutable state
- d) Breaks programs into procedures or routines

Ans: Matches:

- **Functional** → c) Uses functions to avoid mutable state
- **Object-Oriented** → b) Focuses on objects and classes
- **Procedural** → d) Breaks programs into procedures or routines
- **Logic-Based** → a) Uses facts and rules for computation

Q: 3. True or False:

- a) All object-oriented programs are procedural.
- b) Structured programming is a subset of imperative programming.
- c) Functional programming encourages global variables.

Ans: Answers:

- a) **False:** OOP encapsulates state inside objects rather than relying on top-level procedural decomposition.
- b) **True:** Structured programming is a formalized subset of imperative programming that restricts control flow.
- c) **False:** Functional programming strictly avoids mutable state and global variables to prevent side effects.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 5 | SLO 1: Imperative Programming Paradigm (Code: 1051)

Q: 1. What is the main focus of the imperative programming paradigm?

Ans: Answer: The main focus is describing **HOW** a program operates through explicit, step-by-step instructions and statements that directly manipulate the program's memory state over time.

Q: 2. Name two key features of imperative languages.

Ans: 1. **Mutable State & Variables:** Program execution involves tracking and modifying values in named memory locations.
2. **Explicit Control Flow:** Direct ordering of execution using sequential statements, loops, conditional branching, and procedure calls.

Q: 3. Give one example of an imperative programming language.

Ans: Answer: C (or Pascal, Fortran, Java).

Q: 4. How is control flow managed in imperative programming?

Ans: Answer: Control flow is governed explicitly through conditional branching (if/else, switch), iteration constructs (for, while, do-while), and subroutine call/return mechanisms.

Q: 5. Is Java an imperative language? Explain in one sentence.

Ans: Answer: Yes, Java is fundamentally imperative because program logic is implemented through sequenced statements, variable state mutations, and explicit control flow structures within methods.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit I | Session 5 | SLO 2: Programming Paradigm Hierarchy (Code: 1052)

Q: 1. Discuss the transition from Unstructured to Structured and Procedural Imperative Programming.

Ans: Discussion: Early imperative languages relied on unconditional jumps (goto), resulting in fragmented codebases with high maintenance overhead. The structured programming paradigm solved this by enforcing modular single-entry, single-exit constructs (sequence, selection, iteration). Procedural programming further refined this by abstracting repetitive code blocks into parameterized procedures and functions with local scoping.

Q: 2. Provide a Java code example illustrating the imperative paradigm with state mutation.

Ans: Java Implementation:

```
public class ImperativeStateDemo {
    public static void main(String[] args) {
        int total = 0; // Initial state
        int[] numbers = {10, 20, 30, 40};
        for (int i = 0; i < numbers.length; i++) {
            total += numbers[i]; // State mutated step-by-step
        }
        System.out.println("Cumulative Total: " + total);
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 1 | SLO 1: Classes & Objects (Code: 2011)

Q: Activity: Create a Product class with the following attributes: productName (String), price (double), quantity (int). The class should have methods to display product details. Create two objects for the Product class and display the respective product details.

Ans: Java Implementation:

```
class Product {
    String productName;
    double price;
    int quantity;

    Product(String name, double p, int q) {
        this.productName = name;
        this.price = p;
        this.quantity = q;
    }

    void displayDetails() {
        System.out.println("Product Name: " + productName);
        System.out.println("Price: $" + price);
        System.out.println("Quantity: " + quantity);
        System.out.println("Total Value: $" + (price * quantity));
        System.out.println("-----");
    }
}

public class ProductDemo {
    public static void main(String[] args) {
```

```
Product p1 = new Product("Laptop", 1200.50, 5);
Product p2 = new Product("Wireless Mouse", 25.75, 20);
p1.displayDetails();
p2.displayDetails();
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 1 | SLO 2: Constructors, this Keyword & Methods (Code: 2012)

Q: Activity: Design a Box class that implements Default, Parameterized, and Copy Constructors. Include a method to compute volume and demonstrate the use of the 'this' keyword to resolve shadowing.

Ans: Java Implementation:

```
class Box {
    double width, height, depth;

    // 1. Default Constructor
    Box() {
        this.width = 1.0;
        this.height = 1.0;
        this.depth = 1.0;
    }

    // 2. Parameterized Constructor with 'this' reference
    Box(double width, double height, double depth) {
        this.width = width;
        this.height = height;
        this.depth = depth;
    }

    // 3. Copy Constructor
    Box(Box other) {
        this.width = other.width;
        this.height = other.height;
        this.depth = other.depth;
    }

    double volume() {
        return width * height * depth;
    }
}

public class BoxDemo {
    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = new Box(10.0, 5.0, 2.0);
        Box b3 = new Box(b2);

        System.out.println("Default Box Volume: " + b1.volume());
        System.out.println("Parameterized Box Volume: " + b2.volume());
        System.out.println("Copied Box Volume: " + b3.volume());
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 2 | SLO 1: Data Types, Variables & Scope (Code: 2021)

Q: Activity: Implement a Java program demonstrating primitive data types, variable scope, dynamic initialization, and the distinction between local, instance, and static variables.

Ans: Java Implementation:

```
public class VariableScopeDemo {
    // Static (class) variable - single copy per class
    static int classCounter = 0;

    // Instance variable - unique per object
    String objectLabel;

    public VariableScopeDemo(String label) {
        this.objectLabel = label;
        classCounter++;
    }

    public void compute() {
        // Local variable - exists only in method scope
        byte byteVal = 100;
        double dynamicVal = Math.sqrt(byteVal);
        System.out.println("Label: " + objectLabel);
        System.out.println("Dynamic Local Value: " + dynamicVal);
        System.out.println("Class Counter (Static): " + classCounter);
    }

    public static void main(String[] args) {
        VariableScopeDemo d1 = new VariableScopeDemo("Object Alpha");
        d1.compute();
        VariableScopeDemo d2 = new VariableScopeDemo("Object Beta");
        d2.compute();
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 2 | SLO 2: Identifiers, Operators & Access Modifiers (Code: 2022)

Q: Activity: Demonstrate Java access modifiers (private, default, protected, public) and operator precedence with arithmetic, relational, and logical operators in an application.

Ans: Java Implementation:

```
class AccessDemo {
    private int secretKey = 999;    // Accessible only within class
    int defaultVal = 10;           // Package-private
    protected double protectedVal = 55.5; // Subclasses & package
    public String publicMsg = "Open"; // Accessible anywhere

    public int getSecretKey() {
        return secretKey; // Controlled accessor
    }
}

public class OperatorPrecedenceDemo {
    public static void main(String[] args) {
        AccessDemo demo = new AccessDemo();
        System.out.println("Public: " + demo.publicMsg);
        System.out.println("Secret via getter: " + demo.getSecretKey());

        // Operator precedence evaluation
        int a = 10, b = 5, c = 2;
        int res = a + b * c; // Multiplication precedes addition
        boolean check = (a > b) && (b != 0);
        System.out.println("Result of 10 + 5 * 2 = " + res);
        System.out.println("Logical condition (10 > 5 && 5 != 0): " + check);
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 3 | SLO 1: Inheritance & Method Overriding (Code: 2031)

Q: Activity: Implement Single and Multilevel Inheritance representing an Employee hierarchy. Create a superclass Employee and subclasses Manager and Executive. Demonstrate super constructor invocation and method overriding.

Ans: Java Implementation:

```
class Employee {
    String name;
    double basicSalary;

    Employee(String name, double basicSalary) {
        this.name = name;
        this.basicSalary = basicSalary;
    }

    double calculateSalary() {
        return basicSalary;
    }
}

class Manager extends Employee {
    double bonus;

    Manager(String name, double basicSalary, double bonus) {
        super(name, basicSalary); // Super constructor
        this.bonus = bonus;
    }

    @Override
    double calculateSalary() {
        return super.calculateSalary() + bonus;
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Employee emp = new Employee("Alice", 50000);
        Manager mgr = new Manager("Bob", 70000, 15000);
        System.out.println("Employee Salary: " + emp.calculateSalary());
        System.out.println("Manager Salary: " + mgr.calculateSalary());
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 3 | SLO 2: Polymorphism (Compile-time & Runtime) (Code: 2032)

Q: Activity: Write a program demonstrating both Compile-Time Polymorphism (Method Overloading) and Runtime Polymorphism (Method Overriding via Dynamic Method Dispatch) using geometric shapes.

Ans: Java Implementation:

```
class ShapeCalculator {
    // Method Overloading (Compile-time polymorphism)
    double area(double radius) {
        return Math.PI * radius * radius;
    }
    double area(double length, double width) {
        return length * width;
    }
}

class Shape {
    void draw() {
        System.out.println("Drawing generic shape");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing Circle");
    }
}

public class PolymorphismDemo {
    public static void main(String[] args) {
        ShapeCalculator calc = new ShapeCalculator();
        System.out.println("Circle Area: " + calc.area(4.0));
        System.out.println("Rectangle Area: " + calc.area(4.0, 6.0));

        // Dynamic Method Dispatch (Runtime Polymorphism)
        Shape myShape = new Circle();
        myShape.draw();
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 4 | SLO 1: Abstract Classes & Interfaces (Code: 2041)

Q: Activity: Design a banking system illustrating the difference between an Abstract Class and an Interface. Implement an abstract class BankAccount and an interface Transaction.

Ans: Java Implementation:

```
interface Transaction {
    void deposit(double amount);
    void withdraw(double amount);
}

abstract class BankAccount implements Transaction {
    String accountNumber;
    double balance;

    BankAccount(String accNo, double bal) {
        this.accountNumber = accNo;
        this.balance = bal;
    }

    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: $" + amount + ", New Balance: $" + balance);
    }

    abstract void applyInterest();
}

class SavingsAccount extends BankAccount {
    SavingsAccount(String accNo, double bal) {
        super(accNo, bal);
    }

    public void withdraw(double amount) {
        if (balance >= amount) balance -= amount;
    }

    void applyInterest() {
        balance += balance * 0.04;
        System.out.println("Interest applied. Balance: $" + balance);
    }
}

public class BankDemo {
    public static void main(String[] args) {
        SavingsAccount sa = new SavingsAccount("SA-101", 1000.0);
        sa.deposit(500.0);
        sa.applyInterest();
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 4 | SLO 2: Packages & Information Hiding (Code: 2042)

Q: Activity: Demonstrate the concept of package declaration, importing packages, and how access specifiers enforce encapsulation across package boundaries in Java.

Ans: Java Implementation:

```
// File: srm/cse/app/SecurityModule.java
package srm.cse.app;

public class SecurityModule {
    public void grantAccess() {
        System.out.println("Public access granted across packages.");
    }

    protected void internalConfig() {
        System.out.println("Protected config: available in subclasses/package.");
    }
}

// File: ClientApplication.java
import srm.cse.app.SecurityModule;

public class ClientApplication {
    public static void main(String[] args) {
        SecurityModule sec = new SecurityModule();
        sec.grantAccess(); // Accessible
        // sec.internalConfig(); // Compilation error: protected method outside package
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 5 | SLO 1: Exception Handling (try-catch-finally) (Code: 2051)

Q: Activity: Implement a robust exception handling program illustrating try, multiple catch blocks, and the finally block for resource cleanup.

Ans: Java Implementation:

```
public class ExceptionHandlingDemo {
    public static void main(String[] args) {
        try {
            int[] arr = {10, 20, 30};
            int numerator = 100;
            int denominator = 0;
            // Triggers ArithmeticException
            int result = numerator / denominator;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.err.println("Caught Math Exception: Division by zero!");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.err.println("Caught Array Index Exception!");
        } finally {
            System.out.println("Finally block executed: Cleaned up resources.");
        }
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit II | Session 5 | SLO 2: Custom User-Defined Exceptions (Code: 2052)

Q: Activity: Create a custom user-defined exception `InvalidAgeException` to validate candidate eligibility for voting. Use `throw` and `throws` keywords.

Ans: Java Implementation:

```
class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class CustomExceptionDemo {
    static void validateAge(int age) throws InvalidAgeException {
        if (age < 18) {
            throw new InvalidAgeException("Age " + age + " is invalid: Must be 18 or older.");
        }
        System.out.println("Age verified: Eligible to vote!");
    }

    public static void main(String[] args) {
        try {
            validateAge(15);
        } catch (InvalidAgeException e) {
            System.err.println("Custom Exception Caught: " + e.getMessage());
        }

        try {
            validateAge(21);
        } catch (InvalidAgeException e) {
            System.err.println(e.getMessage());
        }
    }
}
```

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit III | Session 1 | SLO 1: Introduction to Concurrent Programming (Code: 3011)

Q: 1. Fill in the blank:

In concurrent programming, multiple _____ can be executed in overlapping time periods.

Ans: Answer: threads (or processes / tasks)

Q: 2. True or False:

A thread is a lightweight process used in concurrent programming.

Ans: Answer: True

Q: 3. Multiple Choice:

Which of the following is commonly used for managing concurrency?

A) Semaphore B) Compiler C) Keyboard D) Monitor

Ans: Answer: A) Semaphore (and D is also a concurrency construct; Semaphore is the standard synchronization primitive).

Q: 4. Fill in the blank:

A _____ condition occurs when two or more threads wait for each other indefinitely.

Ans: Answer: deadlock

Q: 5. True or False:

Concurrency and parallelism refer to the exact same concept.

Ans: Answer: False (Concurrency is dealing with lots of things at once; parallelism is doing lots of things at once simultaneously).

Q: 6. Fill in the blank:

A _____ occurs when two threads simultaneously modify shared data leading to inconsistent states.

Ans: Answer: race condition

Q: 7. True or False:

Threads belonging to the same process share the same memory space.

Ans: Answer: True

Q: 8. Fill in the blank:

Ensuring that multiple threads execute coordinated access to shared variables is called thread _____.

Ans: Answer: synchronization

Q: 9. Multiple Choice:

A mutual exclusion mechanism that allows only one thread into a critical section is called:

A) Mutex B) JVM C) Garbage Collector D) Compiler

Ans: Answer: A) Mutex

Q: 10. Fill in the blank:

A _____ ensures that only one thread accesses a critical section at a time.

Ans: Answer: mutex (or lock / monitor)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit III | Session 1 | SLO 2: Thread Lifecycle and States (Code: 3012)

Q: 1. Fill in the blank:

A thread in Java is created in the _____ state before start() is invoked.

Ans: Answer: NEW

Q: 2. True or False:

Calling run() directly creates a new operating system thread.

Ans: Answer: False (Calling run() executes the method in the caller's thread; start() creates a new thread).

Q: 3. Multiple Choice:

Which method must be called to begin thread execution?

A) run() B) start() C) init() D) execute()

Ans: Answer: B) start()

Q: 4. Fill in the blank:

When a thread is waiting to acquire a monitor lock, it is in the _____ state.

Ans: Answer: BLOCKED

Q: 5. Programming – Output:

```
Thread t = new Thread(() -> System.out.println("Thread running"));  
t.start();
```

Ans: Output: Thread running

Q: 6. True or False:

A terminated thread can be restarted by calling start() again.

Ans: Answer: False (Throws IllegalStateException).

Q: 7. Multiple Choice:

When the run() method finishes executing, the thread transitions to which state?

A) WAITING B) RUNNABLE C) TERMINATED D) BLOCKED

Ans: Answer: C) TERMINATED

Q: 8. Fill in the blank:

The method Thread._____(ms) pauses execution for specified milliseconds.

Ans: Answer: sleep

Q: 9. Identify Error:

```
Thread t = new Thread(); t.start(); t.start();
```

Ans: Answer: Throws IllegalStateException at runtime.

Q: 10. Fill in the blank:

The exact scheduling order of multiple runnable threads is _____.

Ans: Answer: non-deterministic (or determined by OS thread scheduler)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit III | Session 2 | SLO 1: Thread Synchronization & Locks (Code: 3021)

Q: 1. Fill in the blank:

The _____ keyword in Java prevents concurrent access to a shared method or block.

Ans: Answer: synchronized

Q: 2. True or False:

Every Java object has an associated intrinsic lock (monitor).

Ans: Answer: True

Q: 3. Multiple Choice:

Which of the following guarantees atomic compound operations?

A) AtomicInteger B) int C) double D) boolean

Ans: Answer: A) AtomicInteger

Q: 4. Fill in the blank:

A thread acquires an _____ lock when entering a synchronized block.

Ans: Answer: intrinsic (or monitor)

Q: 5. What problem occurs if two threads increment an unsynchronized shared counter simultaneously?

Ans: Answer: Race Condition (Lost Update anomaly).

Q: 6. True or False:

Static synchronized methods lock on the Class object.

Ans: Answer: True

Q: 7. Fill in the blank:

The _____ class in `java.util.concurrent.locks` provides explicit locking mechanisms.

Ans: Answer: ReentrantLock

Q: 8. Fill in the blank:

The _____ keyword ensures visibility of changes to variables across threads.

Ans: Answer: volatile

Q: 9. Identify the issue with: `synchronized(new Object()) { ... }`

Ans: Answer: Ineffective locking because every thread synchronizes on a brand new object instance.

Q: 10. Fill in the blank:

ReentrantLock allows a thread to re-acquire a lock it already holds without causing _____.

Ans: Answer: deadlock

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit III | Session 2 | SLO 2: Inter-Thread Communication (Code: 3022)

Q: 1. Fill in the blank:

The wait(), notify(), and notifyAll() methods belong to the _____ class.

Ans: Answer: Object

Q: 2. True or False:

wait() releases the monitor lock while the thread is waiting.

Ans: Answer: True

Q: 3. Multiple Choice:

Which method wakes up all threads waiting on an object's monitor?

A) notify() B) resume() C) notifyAll() D) wakeAll()

Ans: Answer: C) notifyAll()

Q: 4. Fill in the blank:

wait() must always be called inside a _____ context.

Ans: Answer: synchronized

Q: 5. What exception is thrown if wait() is called without holding the monitor lock?

Ans: Answer: IllegalMonitorStateException

Q: 6. True or False:

Thread.sleep() releases locks while sleeping.

Ans: Answer: False (sleep() holds locks, whereas wait() releases the lock).

Q: 7. Fill in the blank:

To guard against spurious wakeups, wait() should always be placed inside a _____ loop.

Ans: Answer: while

Q: 8. Multiple Choice:

Which classical synchronization problem demonstrates inter-thread communication?

A) Producer-Consumer B) QuickSort C) Binary Search D) Bubble Sort

Ans: Answer: A) Producer-Consumer

Q: 9. Identify the bug: Calling notify() outside synchronized block.

Ans: Answer: Results in IllegalMonitorStateException at runtime.

Q: 10. Fill in the blank:

notify() wakes up an _____ thread waiting on the object monitor.

Ans: Answer: arbitrarily chosen (single)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit III | Session 3 | SLO 1: Thread Pooling and Executors (Code: 3031)

Q: 1. Fill in the blank:

The _____ interface represents an asynchronous execution service in `java.util.concurrent`.

Ans: Answer: `ExecutorService`

Q: 2. True or False:

Creating new threads on demand for every task introduces significant performance overhead.

Ans: Answer: True

Q: 3. Multiple Choice:

Which factory method creates a pool with a set number of threads?

A) `newCachedThreadPool()` B) `newFixedThreadPool()` C) `newSingleThreadExecutor()` D) `newScheduledThreadPool()`

Ans: Answer: B) `newFixedThreadPool()`

Q: 4. Fill in the blank:

Reusing worker threads reduces the overhead of thread _____ and destruction.

Ans: Answer: creation (allocation)

Q: 5. Programming – What happens if `executor.shutdown()` is not called?

Ans: Answer: The JVM may not terminate because active non-daemon pool threads prevent shutdown.

Q: 6. True or False:

`ExecutorService.shutdownNow()` attempts to stop all actively executing tasks.

Ans: Answer: True

Q: 7. Fill in the blank:

The core implementation class behind Executors factory methods is _____.

Ans: Answer: `ThreadPoolExecutor`

Q: 8. Multiple Choice:

Which method is used to submit a task to an `ExecutorService`?

A) `run()` B) `execute()` / `submit()` C) `start()` D) `begin()`

Ans: Answer: B) `execute()` / `submit()`

Q: 9. Fill in the blank:

A pool queue that holds tasks waiting for available threads is typically a _____.

Ans: Answer: `BlockingQueue`

Q: 10. Fill in the blank:

Thread pools prevent resource exhaustion by limiting the maximum number of _____ threads.

Ans: Answer: concurrent (active)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit III | Session 3 | SLO 2: Callable and Future (Code: 3032)

Q: 1. Fill in the blank:

Unlike Runnable, the _____ interface can return a computed result.

Ans: Answer: Callable (java.util.concurrent.Callable)

Q: 2. True or False:

Callable's call() method can throw checked exceptions.

Ans: Answer: True

Q: 3. Multiple Choice:

What method does the Callable interface define?

A) run() B) execute() C) call() D) compute()

Ans: Answer: C) call()

Q: 4. Fill in the blank:

A _____ object represents the pending result of an asynchronous computation.

Ans: Answer: Future

Q: 5. What does the Future.get() method do if the task is still calculating?

Ans: Answer: It blocks the calling thread until computation finishes.

Q: 6. True or False:

Future.isDone() returns true if the task completed normally, threw an exception, or was cancelled.

Ans: Answer: True

Q: 7. Multiple Choice:

Which method cancels task execution?

A) stop() B) abort() C) cancel() D) terminate()

Ans: Answer: C) cancel()

Q: 8. Fill in the blank:

Future.get(timeout, unit) throws _____ if the timeout expires before completion.

Ans: Answer: TimeoutException

Q: 9. Identify Error: Callable c = () -> { return 5; }; Thread t = new Thread(c);

Ans: Answer: Thread constructor accepts Runnable, not Callable directly (use FutureTask or ExecutorService).

Q: 10. Fill in the blank:

Callable is a parameterized functional interface with generic type _____.

Ans: Answer: Callable<V> (or <T>)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit III | Session 4 | SLO 1: Thread Priorities and Daemon Threads (Code: 3041)

Q: 1. Fill in the blank:

The default priority of a thread in Java is _____.

Ans: Answer: 5 (Thread.NORM_PRIORITY)

Q: 2. True or False:

Daemon threads continue running even after all user threads have finished execution.

Ans: Answer: False (JVM terminates automatically when all user threads finish).

Q: 3. Multiple Choice:

Which method is used to set the priority of a thread?

A) setOrder() B) setLevel() C) setPriority() D) setThreadLevel()

Ans: Answer: C) setPriority()

Q: 4. Fill in the blank:

The priority range for threads in Java is from _____ to _____.

Ans: Answer: 1 (MIN_PRIORITY) to 10 (MAX_PRIORITY)

Q: 5. Programming – Find output:

```
Thread t = new Thread(() -> System.out.println(Thread.currentThread().isDaemon()));  
t.setDaemon(true); t.start();
```

Ans: Output: A) true

Q: 6. True or False:

Calling setDaemon(true) on a thread after start() has been called will throw an exception.

Ans: Answer: True (IllegalThreadStateException)

Q: 7. Multiple Choice:

Which thread type is typically used for background services like garbage collection?

A) Foreground B) User thread C) Daemon thread D) High-priority

Ans: Answer: C) Daemon thread

Q: 8. Fill in the blank:

The method _____ returns whether a thread is a daemon.

Ans: Answer: isDaemon()

Q: 9. Identify Error:

```
Thread t = new Thread(() -> System.out.println("Running")); t.start(); t.setDaemon(true);
```

Ans: Answer: C) IllegalStateException (setDaemon must precede start()).

Q: 10. Fill in the blank:

Threads with higher priority are _____ likely to be scheduled before lower priority threads.

Ans: Answer: more

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit III | Session 4 | SLO 2: Runnable Interface (Code: 3042)

Q: 1. Fill in the blank:

In Java, a class implements the _____ interface to create a thread using Runnable.

Ans: Answer: Runnable

Q: 2. True or False:

The run() method must be defined when implementing the Runnable interface.

Ans: Answer: True

Q: 3. Multiple Choice:

Which is the correct way to create a thread using Runnable?

A) new Thread(new MyRunnable()).start(); B) new Runnable().start(); C) new Thread().run(); D) new MyRunnable().start();

Ans: Answer: A) new Thread(new MyRunnable()).start();

Q: 4. Fill in the blank:

The Runnable interface belongs to the java.lang _____.

Ans: Answer: package

Q: 5. True or False:

The Runnable interface can be used to create multiple threads sharing the same task.

Ans: Answer: True

Q: 6. Multiple Choice:

What keyword is used when a class uses the Runnable interface?

A) extends B) implements C) imports D) inherits

Ans: Answer: B) implements

Q: 7. Fill in the blank:

The run() method of a Runnable object is invoked by the _____ class.

Ans: Answer: Thread

Q: 8. True or False:

Runnable allows better separation of thread creation and task definition.

Ans: Answer: True

Q: 9. Multiple Choice:

Which of the following is an advantage of using Runnable over extending Thread?

A) Allows multiple inheritance B) Allows sharing the same task between threads C) Direct JVM access D) Overriding main

Ans: Answer: B) Allows sharing the same task between threads (and A is also a primary benefit).

Q: 10. Fill in the blank:

A thread is started by calling the _____ method on the Thread object.

Ans: Answer: start()

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit III | Session 5 | SLO 1: Declarative Programming in Databases (Code: 3051)

Q: 1. Fill in the blank:

In declarative programming, the focus is on _____ rather than how the task is performed.

Ans: Answer: what (the desired result)

Q: 2. True or False:

SQL is an example of declarative programming language used in databases.

Ans: Answer: True

Q: 3. Multiple Choice:

Which of the following best represents a declarative SQL statement?

A) Use for-loop to iterate B) SELECT name FROM students WHERE grade > 90; C) Stored procedure loop D) Cursor reading

Ans: Answer: B) SELECT name FROM students WHERE grade > 90;

Q: 4. Fill in the blank:

Declarative programming abstracts away the _____ of data access.

Ans: Answer: internal implementation details (procedural algorithms)

Q: 5. SQL Task – Find Output:

Table: Students (1 Alice 85, 2 Bob 92, 3 Clara 88)

Query: SELECT name FROM Students WHERE grade > 90;

Ans: Output: B) Bob

Q: 6. True or False:

In declarative programming, the database engine determines the most efficient way to execute a query.

Ans: Answer: True (via the Query Optimizer and Execution Plan).

Q: 7. Multiple Choice:

Which of these is NOT a feature of declarative programming in databases?

A) Focus on result B) Automatic optimization C) Explicit control flow D) Use of queries

Ans: Answer: C) Explicit control flow

Q: 8. Fill in the blank:

The WHERE clause in SQL is used to _____ records in declarative queries.

Ans: Answer: filter

Q: 9. SQL Task – Identify Error:

SELECT * FROM students WHERE grade => 90;

Ans: Answer: C) Invalid operator (syntax error: operator must be >= not =>).

Q: 10. Fill in the blank:

Declarative programming helps developers express logic without describing its _____.

Ans: Answer: execution control flow (steps)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

School of Computing — Department of Computer Science & Engineering

Student Name: Sautrik Roy	Reg No: RA2511003010052
Course: 21CSC203P — Advanced Programming Practice	Department: Computer Science and Engineering

Unit III | Session 5 | SLO 2: Architecture of JDBC in Java (Code: 3052)

Q: 1. Fill in the blank:

The JDBC API provides a _____-independent way to access databases from Java.

Ans: Answer: database (or vendor)

Q: 2. True or False:

The DriverManager class is responsible for managing a list of database drivers.

Ans: Answer: True

Q: 3. Multiple Choice:

Which component acts as the interface between a Java application and the database driver?

A) Connection B) ResultSet C) Statement D) DriverManager

Ans: Answer: D) DriverManager

Q: 4. Fill in the blank:

The method used to register a JDBC driver class manually is Class.forName("_____").

Ans: Answer: driver class name (e.g., com.mysql.cj.jdbc.Driver)

Q: 5. Programming – Output:

```
Connection con = DriverManager.getConnection(url, user, pass);
```

```
if (con != null) System.out.println("Connected");
```

Ans: Output: Connected (assuming valid database connection).

Q: 6. True or False:

The ResultSet object allows forward and backward navigation by default.

Ans: Answer: False (Default ResultSet is forward-only).

Q: 7. Multiple Choice:

Which interface is used to execute static SQL queries?

A) Statement B) CallableStatement C) PreparedStatement D) Connection

Ans: Answer: A) Statement

Q: 8. Fill in the blank:

The method executeQuery() of Statement returns a _____ object.

Ans: Answer: ResultSet

Q: 9. Programming – Identify Error:

```
Statement stmt = con.createStatement();
```

```
ResultSet rs = stmt.executeUpdate("SELECT * FROM users");
```

Ans: Answer: B) executeQuery should be used (executeUpdate returns an int, not ResultSet).

Q: 10. Fill in the blank:

The JDBC architecture consists of two main layers: the JDBC API and the JDBC _____.

Ans: Answer: Driver API